

Documento Técnico del Proyecto: Gestor de Tareas Android

1. Descripción del proyecto

- **Objetivo:**

El objetivo de este proyecto es desarrollar una aplicación móvil para la gestión simple de tareas, permitiendo al usuario crear, visualizar y almacenar tareas localmente en su dispositivo Android.

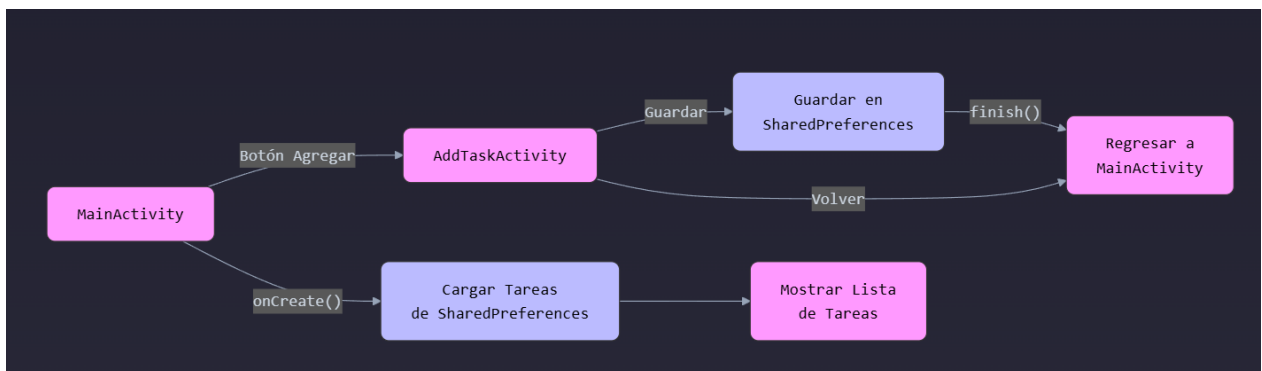
- **Funcionalidades principales:**

- Visualización de una lista de tareas guardadas.
- Adición de nuevas tareas con nombre y descripción.
- Almacenamiento persistente de las tareas mediante SharedPreferences.
- Navegación simple entre dos pantallas: principal y formulario.

- **La aplicación se desarrolló en dos versiones:**

- Enfoque tradicional con XML.
- Enfoque moderno con Jetpack Compose.

2. Diagrama del flujo de navegación



Descripción del flujo:

- Al iniciar la aplicación, se muestra la pantalla principal con la lista de tareas.
- El usuario puede presionar el botón “Agregar Nueva Tarea” para ir a la pantalla del formulario.

3. En el formulario puede:

- Ingresar el nombre y descripción de una tarea y presionar "Guardar", lo cual: Guarda la tarea en SharedPreferences.
- Finaliza la actividad y regresa automáticamente a la pantalla principal.
- O presionar "Volver", lo que también regresa sin guardar.

4. Explicación breve del código

- Arquitectura del proyecto:

El proyecto está compuesto por dos actividades (MainActivity y FormActivity) y dos diseños (activity_main.xml y activity_form.xml) en la versión XML.

En Compose, se reemplazaron estos layouts por funciones @Composable declarativas que manejan tanto el diseño como la lógica.

- Manejo de datos:

Las tareas se almacenan como un arreglo JSON en SharedPreferences bajo la clave "task_list".

MainActivity carga y muestra la lista al iniciar y al volver de FormActivity.

- Principales funciones:

loadTasks() en MainActivity: Carga las tareas desde almacenamiento local.

onClick de btnSaveTask en FormActivity: Agrega una nueva tarea y la guarda.

En Compose, LaunchedEffect simula el comportamiento de onResume, recargando las tareas al volver a la pantalla principal.

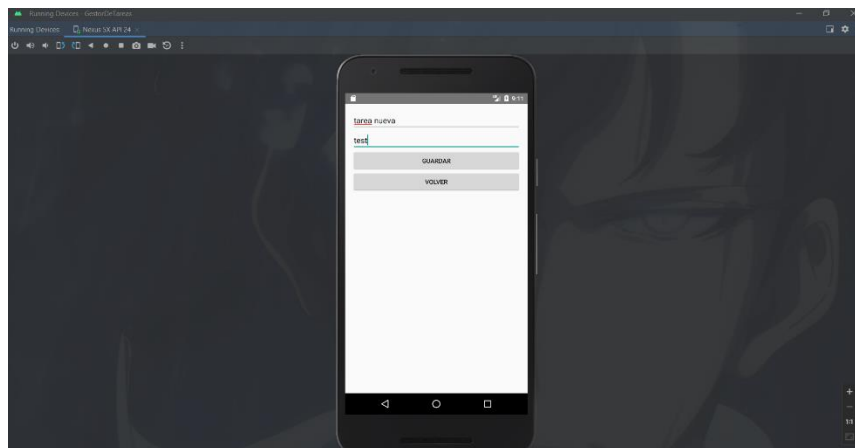
remember y mutableStateOf se utilizan para manejar el estado de la interfaz en tiempo real.

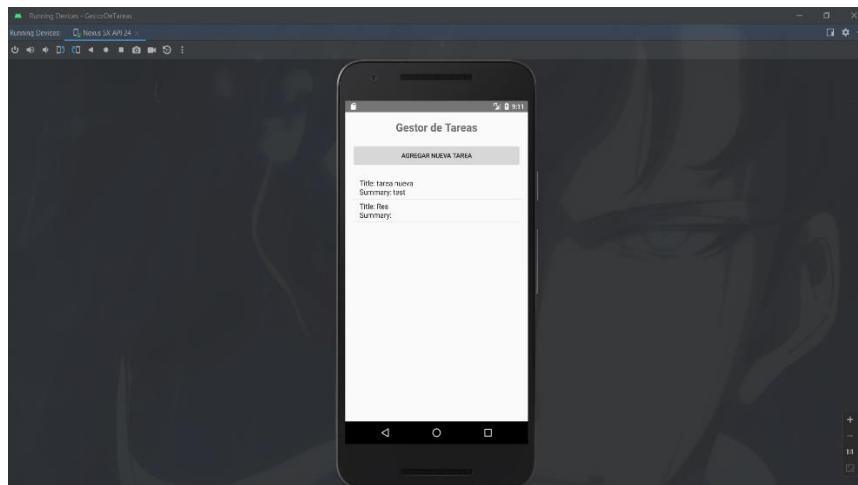
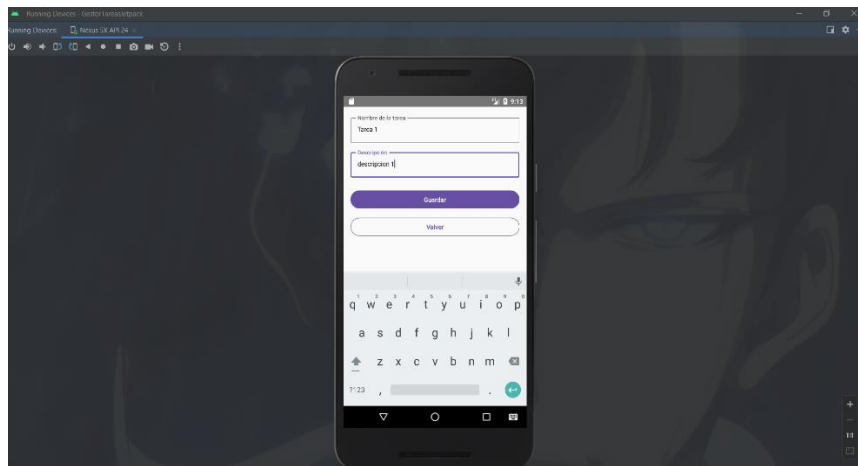
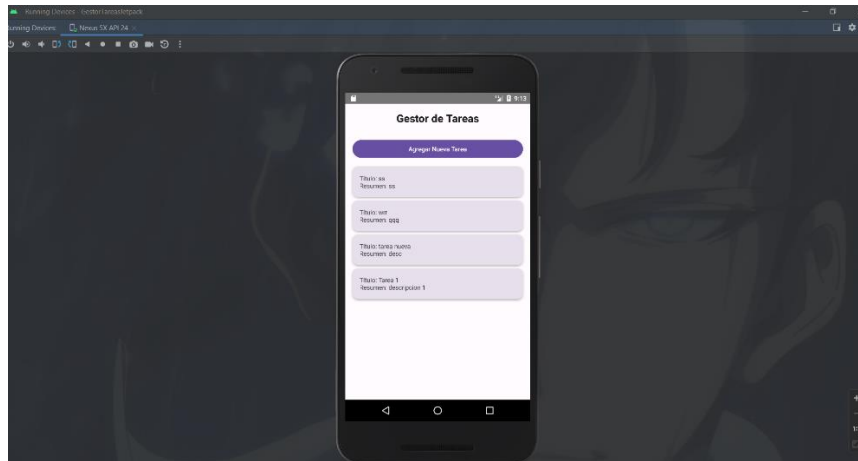
5. Conclusiones

Durante el desarrollo del proyecto se enfrentaron desafíos tanto en la versión XML como en Jetpack Compose. En XML, el manejo del estado requería recargar manualmente los datos usando `onResume`, mientras que en Compose fue necesario comprender y aplicar correctamente `LaunchedEffect` para replicar ese comportamiento. Además, el uso de `SharedPreferences` implicó gestionar manualmente la serialización y deserialización de objetos en JSON, lo cual podría complejizarse a medida que aumente la estructura de datos. No obstante, este proceso también permitió obtener aprendizajes significativos: Jetpack Compose facilitó la creación de interfaces más limpias y reactivas, eliminando la necesidad de `findViewById` y mejorando la legibilidad del código. Su enfoque declarativo y la separación clara entre estado y UI representan un paso natural hacia arquitecturas modernas como MVVM, promoviendo un desarrollo más escalable y mantenible.

6. Anexos

- [Repositorio de código](#)
- Imágenes de aplicación funcional





- Muestras de Código

```

package com.everbravo.gestordetareas

package com.everbravo.gestordetareas

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.widget.AdapterView
import android.widget.Button
import android.widget.ListView
import androidx.activity.ComponentActivity
import org.json.JSONArray

/**
 * MainActivity is the entry point of the Task Manager app.
 * It displays a list of saved tasks and provides a button to add new ones.
 *
 * Tasks are stored in SharedPreferences in JSON format and loaded every time the activity
 * resumes.
 * @author Ever Bravo
 * @version 1.0
 */
class MainActivity : ComponentActivity() {

    /** ListView used to display the list of tasks. */
    private lateinit var listView: ListView

    /** Adapter to bind the task list to the ListView. */
    private lateinit var adapter: ArrayAdapter<String>

    /** Mutable list of task descriptions to be shown in the UI. */
    private val taskList = mutableListOf<String>()

    /**
     * Called when the activity is first created.
     * Sets up the layout, initializes the ListView and adapter,
     * and configures the Add Task button to launch FormActivity.
     *
     * @param savedInstanceState the previously saved state of the activity, if any
     */
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        listView = findViewById(R.id.listTasks)
        val btnAddTask = findViewById<Button>(R.id.btnAddTask)

        adapter = ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, taskList)
        listView.adapter = adapter

        btnAddTask.setOnClickListener {
            startActivity(Intent<String>(this, FormActivity::class.java))
        }
    }

    /**
     * Called when the activity becomes visible to the user.
     * Reloads the task list from SharedPreferences.
     */
    override fun onResume() {
        super.onResume()
        loadTasks()
    }

    /**
     * Loads the list of tasks from SharedPreferences.
     * Clears the current list, reads the JSON array, and updates the ListView.
     */
    private fun loadTasks() {
        taskList.clear()
        val sharedPreferences = getSharedPreferences("Tasks", Context.MODE_PRIVATE)
        val json = sharedPreferences.getString("task_list", "[ ]")
        val jsonArray = JSONArray(json)

        for (i in 0 until jsonArray.length()) {
            val task = jsonArray.getJSONObject(i)
            val name = task.getString("name")
            val description = task.getString("description")
            taskList.add("Title: $name\nSummary: $description")
        }

        adapter.notifyDataSetChanged()
    }

    /**
     * Called when the activity is no longer visible to the user.
     * This method clears the saved task list from SharedPreferences
     * to ensure that the task list is deleted when the activity stops.
     */
    override fun onStop() {
        super.onStop()
        val sharedPreferences = getSharedPreferences("Tasks", Context.MODE_PRIVATE)
        sharedPreferences.edit().remove("task_list").apply()
    }
}

```

```

package com.everbravo.gestordetareas

import android.content.Context
import android.content.SharedPreferences
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.activity.ComponentActivity
import org.json.JSONArray
import org.json.JSONObject

/**
 * FormActivity allows the user to input and save a new task.
 * It includes fields for the task name and description, and buttons to save the task or go back.
 *
 * The task is stored in SharedPreferences as a JSON array. Each new task is appended to the existing
 * list.
 *
 * @author Ever Bravo
 * @version 1.0
 */
class FormActivity : ComponentActivity() {

    /**
     * Called when the activity is starting. Initializes the layout, input fields,
     * and buttons for saving or canceling task creation.
     *
     * @param savedInstanceState the previously saved state of the activity, if any
     */
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_form)

        val etTaskName = findViewById<EditText>(R.id.etTaskName)
        val etTaskDescription = findViewById<EditText>(R.id.etTaskDescription)
        val btnSaveTask = findViewById<Button>(R.id.btnSaveTask)
        val btnBack = findViewById<Button>(R.id.btnBack)

        val sharedPreferences: SharedPreferences = getSharedPreferences("Tasks", Context.MODE_PRIVATE)

        /**
         * Saves the entered task to SharedPreferences as a JSON object.
         * Appends the new task to the existing task list and finishes the activity.
         */
        btnSaveTask.setOnClickListener {
            val name = etTaskName.text.toString()
            val description = etTaskDescription.text.toString()

            val taskJson = sharedPreferences.getString("task_list", "[ ]")
            val taskArray = JSONArray(taskJson)

            val newTask = JSONObject()
            newTask.put("name", name)
            newTask.put("description", description)

            taskArray.put(newTask)

            sharedPreferences.edit()
                .putString("task_list", taskArray.toString())
                .apply()

            finish()
        }

        /**
         * Finishes the activity and returns to the previous screen without saving.
         */
        btnBack.setOnClickListener {
            finish()
        }
    }
}

```